Assignment 8

Question 1

Classify SMS messages as:
- spam: (1)
- ham: (0)

Data Description

Column	Meaning
label	spam / ham
text	SMS message content

There are 5500 messages

Part A (Data Preprocessing and Exploration)
1. Load the SMS Spam Collection dataset (spam.csv)
2. Convert label: "spam" → 1, "ham" → 0
3. Text preprocessing:
   - Lowercase
   - Remove punctuation
   - Remove stopwords
4. Convert text to numeric feature vectors using TF-IDF vectorizer
5. Train–test split (80/20)
6. Show class distribution

Part B (Weak Learner Baseline)

Train a Decision Stump: `DecisionTreeClassifier(max_depth: 1)`

Report:
- Train accuracy
- Test accuracy
- Confusion matrix
- Comment briefly on why stump performance is weak on high-dimensional text data

Part C (Manual AdaBoost, T = 15)

Implement AdaBoost manually and after each iteration print:
- Iteration number
- Misclassified sample indices
- Weights of misclassified samples
- Alpha value

Then update and normalize weights.

Also produce:
- Plot: iteration vs weighted error
- Plot: iteration vs alpha

Final report:
- Train accuracy
- Test accuracy
- Confusion matrix
- Short interpretation of weight evolution

Part D (Sklearn AdaBoost)

Train:

```
AdaBoostClassifier(
  base_estimator: DecisionTreeClassifier(max_depth: 1),
  n_estimators: 100,
  learning_rate: 0.6
)
```

Solution

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import string
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

pd.options.mode.chained_assignment = None

DATA_URL = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/
data/sms.tsv"

df = pd.read_csv(DATA_URL, sep="\t", header=None, names=["label", "message"],
encoding="latin-1")

df["label"] = df["label"].map({"ham": 0, "spam": 1})

stopwords_df = pd.read_csv("https://raw.githubusercontent.com/stopwords-iso/stopwords-en/
master/stopwords-en.txt", header=None)
stopwords = set(stopwords_df[0].values)


def preprocess_text(text):
    """Cleans text by lowercasing, removing punctuation, and stopwords."""
    text = text.lower()
    text = text.translate(str.maketrans("", "", string.punctuation))
    text = " ".join([word for word in text.split() if word not in stopwords])
    return text


df["cleaned_message"] = df["message"].apply(preprocess_text)

X_text = df["cleaned_message"]
y = df["label"]

X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text, y, test_size=0.2, random_state=42, stratify=y
)

vectorizer = TfidfVectorizer()
X_train = vectorizer.fit_transform(X_train_text)
X_test = vectorizer.transform(X_test_text)


def evaluate_model(model, X_train, y_train, X_test, y_test, name):
```

```
    """Evaluates and prints performance metrics."""
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    train_acc = accuracy_score(y_train, y_train_pred)
    test_acc = accuracy_score(y_test, y_test_pred)
    cm = confusion_matrix(y_test, y_test_pred)

    print(f"Train Accuracy: {train_acc:.4f}")
    print(f"Test Accuracy:  {test_acc:.4f}")
    print("\nConfusion Matrix (Test Set):\n", cm)
    return train_acc, test_acc, cm


stump = DecisionTreeClassifier(max_depth=1, random_state=42)
stump.fit(X_train, y_train)

evaluate_model(stump, X_train, y_train, X_test, y_test, "Decision Stump")

y_train_signed = np.where(y_train == 0, -1, 1)
y_test_signed = np.where(y_test == 0, -1, 1)

N = X_train.shape[0]
T = 15
D = np.full(N, 1 / N)
H_final = np.zeros(N)

error_history = []
alpha_history = []
model_history = []

for t in range(T):
    print(f"\n[Iteration {t + 1}]")

    h_t = DecisionTreeClassifier(max_depth=1, random_state=t)
    h_t.fit(X_train, y_train, sample_weight=D)

    y_pred_01 = h_t.predict(X_train)

    y_pred_signed = np.where(y_pred_01 == 0, -1, 1)

    misclassified_indices = np.where(y_pred_signed != y_train_signed)[0]

    epsilon_t = np.sum(D[misclassified_indices])

    if epsilon_t == 0:
        print("Weighted error is 0. Stopping AdaBoost early.")
        break
    if epsilon_t >= 0.5:
        print(f"Weighted error is {epsilon_t:.4f}. Stump is too weak. Stopping.")
        break

    alpha_t = 0.5 * np.log((1 - epsilon_t) / epsilon_t)

    H_final += alpha_t * y_pred_signed

    print(f"Weighted Error (ε): {epsilon_t:.4f}")
    print(f"Alpha (α): {alpha_t:.4f}")

    print(
```

```
        f"Misclassified samples ({len(misclassified_indices)} total):
{misclassified_indices[:5]}..."
    )
    print(
        f"Initial weights of these samples: {D[misclassified_indices[:5]].round(6)}..."
    )

    D *= np.exp(-alpha_t * y_train_signed * y_pred_signed)
    D /= np.sum(D)

    error_history.append(epsilon_t)
    alpha_history.append(alpha_t)
    model_history.append((alpha_t, h_t))  # Store (alpha, stump model) tuple


def manual_ada_predict(X, model_history):
    """Aggregates predictions from all weak learners."""
    N_samples = X.shape[0]
    final_pred_signed = np.zeros(N_samples)

    for alpha, h_t in model_history:
        y_pred_01 = h_t.predict(X)
        y_pred_signed = np.where(y_pred_01 == 0, -1, 1)
        final_pred_signed += alpha * y_pred_signed

    final_pred_01 = np.where(final_pred_signed > 0, 1, 0)
    return final_pred_01


y_manual_pred_train = manual_ada_predict(X_train, model_history)
y_manual_pred_test = manual_ada_predict(X_test, model_history)

manual_train_acc = accuracy_score(y_train, y_manual_pred_train)
manual_test_acc = accuracy_score(y_test, y_manual_pred_test)
manual_cm = confusion_matrix(y_test, y_manual_pred_test)

print(f"Train Accuracy: {manual_train_acc:.4f}")
print(f"Test Accuracy:  {manual_test_acc:.4f}")
print("\nConfusion Matrix (Test Set):\n", manual_cm)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
iterations = np.arange(1, len(error_history) + 1)

ax1.plot(iterations, error_history, marker="o", linestyle="-", color="red")
ax1.set_title("Iteration vs Weighted Error (ε)")
ax1.set_xlabel("Iteration")
ax1.set_ylabel("Weighted Error (ε)")
ax1.grid(True)

ax2.plot(iterations, alpha_history, marker="o", linestyle="-", color="blue")
ax2.set_title("Iteration vs Alpha (α)")
ax2.set_xlabel("Iteration")
ax2.set_ylabel("Alpha (α) / Classifier Confidence")
ax2.grid(True)

plt.tight_layout()
plt.show()


ada_clf = AdaBoostClassifier(
```

```
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=100,
    learning_rate=0.6,
    random_state=42,
    algorithm="SAMME",
)

ada_clf.fit(X_train, y_train)

sklearn_train_acc, sklearn_test_acc, sklearn_cm = evaluate_model(
    ada_clf, X_train, y_train, X_test, y_test, "Sklearn AdaBoost (100 Est.)"
)

print(f"Manual AdaBoost Test Acc (15 rounds): {manual_test_acc:.4f}")
print(f"Sklearn AdaBoost Test Acc (100 rounds): {sklearn_test_acc:.4f}")
```

Output

```
  Train Accuracy: 0.8892
Test Accuracy:  0.8897

Confusion Matrix (Test Set):
 [[965   1]
 [122  27]]

[Iteration 1]
Weighted Error (ε): 0.1108
Alpha (α): 1.0411
Misclassified samples (494 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.000224 0.000224 0.000224 0.000224 0.000224]...

[Iteration 2]
Weighted Error (ε): 0.4230
Alpha (α): 0.1552
Misclassified samples (509 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.001012 0.001012 0.001012 0.001012 0.001012]...

[Iteration 3]
Weighted Error (ε): 0.4304
Alpha (α): 0.1401
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000109 0.000109 0.000109 0.000109 0.000109]...

[Iteration 4]
Weighted Error (ε): 0.4317
Alpha (α): 0.1374
Misclassified samples (531 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.00105 0.00105 0.00105 0.00105 0.00105]...

[Iteration 5]
Weighted Error (ε): 0.4404
Alpha (α): 0.1198
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000112 0.000112 0.000112 0.000112 0.000112]...

[Iteration 6]
Weighted Error (ε): 0.4332
Alpha (α): 0.1344
Misclassified samples (550 total): [ 9 15 31 39 49]...
Initial weights of these samples: [0.001087 0.001087 0.001087 0.001087 0.001087]...
```

```
[Iteration 7]
Weighted Error (ε): 0.4421
Alpha (α): 0.1164
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000112 0.000112 0.000112 0.000112 0.000112]...

[Iteration 8]
Weighted Error (ε): 0.4379
Alpha (α): 0.1248
Misclassified samples (509 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.001124 0.001124 0.000859 0.001124 0.001124]...

[Iteration 9]
Weighted Error (ε): 0.4448
Alpha (α): 0.1109
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000113 0.000113 0.000113 0.000113 0.000113]...

[Iteration 10]
Weighted Error (ε): 0.4503
Alpha (α): 0.0998
Misclassified samples (509 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.001156 0.001156 0.000883 0.001156 0.001156]...

[Iteration 11]
Weighted Error (ε): 0.4548
Alpha (α): 0.0907
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000115 0.000115 0.000115 0.000115 0.000115]...

[Iteration 12]
Weighted Error (ε): 0.4481
Alpha (α): 0.1041
Misclassified samples (531 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.001177 0.001177 0.000899 0.001177 0.001177]...

[Iteration 13]
Weighted Error (ε): 0.4535
Alpha (α): 0.0933
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000115 0.000115 0.000115 0.000115 0.000115]...

[Iteration 14]
Weighted Error (ε): 0.4519
Alpha (α): 0.0966
Misclassified samples (553 total): [ 9 22 49 50 59]...
Initial weights of these samples: [0.001201 0.000918 0.001201 0.00015  0.001201]...

[Iteration 15]
Weighted Error (ε): 0.4562
Alpha (α): 0.0879
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000115 0.000115 0.000115 0.000115 0.000115]...
Train Accuracy: 0.8923
Test Accuracy:  0.8933

Confusion Matrix (Test Set):
 [[965   1]
 [118  31]]
/home/himanshu/assignments/machine_learning/assignment_08/1.py:181: UserWarning:
```

```
FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
/home/himanshu/assignments/.venv/lib/python3.11/site-packages/sklearn/ensemble/
_weight_boosting.py:519: FutureWarning: The parameter 'algorithm' is deprecated in 1.6 and
has no effect. It will be removed in version 1.8.
  warnings.warn(
Train Accuracy: 0.9055
Test Accuracy:  0.9022

Confusion Matrix (Test Set):
 [[964   2]
 [107  42]]
Manual AdaBoost Test Acc (15 rounds): 0.8933
Sklearn AdaBoost Test Acc (100 rounds): 0.9022
```

QUESTION 2

Heart Disease Prediction using AdaBoost

Dataset Description You will use the UCI Heart Disease dataset (available in `sklearn.datasets`). This dataset contains patient medical attributes used to predict the presence of heart disease.

Feature	Meaning
Age	Patient age
Sex	Gender (1 = male, 0 = female)
Cp	Chest pain type (0–3)
Trestbps	Resting blood pressure
Chol	Serum cholesterol (mg/dl)
Fbs	Fasting blood sugar >120 mg/dl (1/0)
Restecg	Resting ECG results
Thalach	Maximum heart rate achieved
Exang	Exercise-induced angina (1/0)
Oldpeak	ST depression induced by exercise
Slope	Slope of peak exercise ST segment
Ca	Number of major vessels (0–3)
Thal	Thallium stress test result (0–3)

Target: 1 = Heart disease present 0 = No heart disease

Part A — Baseline Model (Weak Learner)

1. Load and preprocess the dataset (handle categorical features, scaling if needed)
2. Train a single Decision Stump (`max_depth = 1`)
3. Report:

- Training & test accuracy
- Confusion matrix
- Classification report

1. Discuss shortcomings of using only one stump

Part B — Train AdaBoost

1. Train `AdaBoostClassifier` using decision stumps as base learners

2. Use:

- `n_estimators = [5, 10, 25, 50, 100]`
- `learning_rate = [0.1, 0.5, 1.0]`

1. For each combination:

- Train model
- Compute test accuracy

1. Plot:

- **n_estimators** vs **accuracy** for each **learning_rate**

1. Identify the best configuration (highest test accuracy)

Part C — Misclassification Pattern

1. For the best model, collect weak-learner errors and sample weights at each iteration
2. Plot:

- Weak learner error vs iteration
- Sample weight distribution after final boosting stage

1. Explain:

- Which samples receive the highest weights?
- Why does AdaBoost focus more on them?

Part D — Visual Explainability

1. Plot feature importances from AdaBoost
2. Identify the top 5 most important features
3. Explain medically why these features may be strong predictors of heart disease

Solution

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

RANDOM_STATE = 42

try:
    data = pd.read_csv(
        "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/
processed.cleveland.data",
        header=None,
        na_values="?",
    )
except Exception:
    print("Could not load data from URL. Using dummy structure.")
    data = pd.DataFrame(np.random.randint(0, 100, size=(303, 14)), columns=range(14))
    data[13] = np.where(data[13] > 40, 1, 0)  # Create dummy target

cols = [
```

```
    "age",
    "sex",
    "cp",
    "trestbps",
    "chol",
    "fbs",
    "restecg",
    "thalach",
    "exang",
    "oldpeak",
    "slope",
    "ca",
    "thal",
    "target",
]
data.columns = cols

data["target"] = data["target"].apply(lambda x: 1 if x > 0 else 0)

data = data.dropna()

X = data.drop("target", axis=1)
y = data["target"]

numeric_features = ["age", "trestbps", "chol", "thalach", "oldpeak"]
categorical_features = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal"]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            categorical_features,
        ),
    ],
    remainder="passthrough",
)

X_processed = preprocessor.fit_transform(X)
feature_names = preprocessor.get_feature_names_out()

X_train, X_test, y_train, y_test = train_test_split(
    X_processed, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

print(f"Processed Data Shapes: Train {X_train.shape}, Test {X_test.shape}")


stump = DecisionTreeClassifier(max_depth=1, random_state=RANDOM_STATE)
stump.fit(X_train, y_train)

y_train_pred = stump.predict(X_train)
y_test_pred = stump.predict(X_test)

print(f"Training Accuracy: {accuracy_score(y_train, y_train_pred):.4f}")
print(f"Test Accuracy:     {accuracy_score(y_test, y_test_pred):.4f}")
print("\nConfusion Matrix (Test):\n", confusion_matrix(y_test, y_test_pred))
print("\nClassification Report (Test):\n", classification_report(y_test, y_test_pred))
```

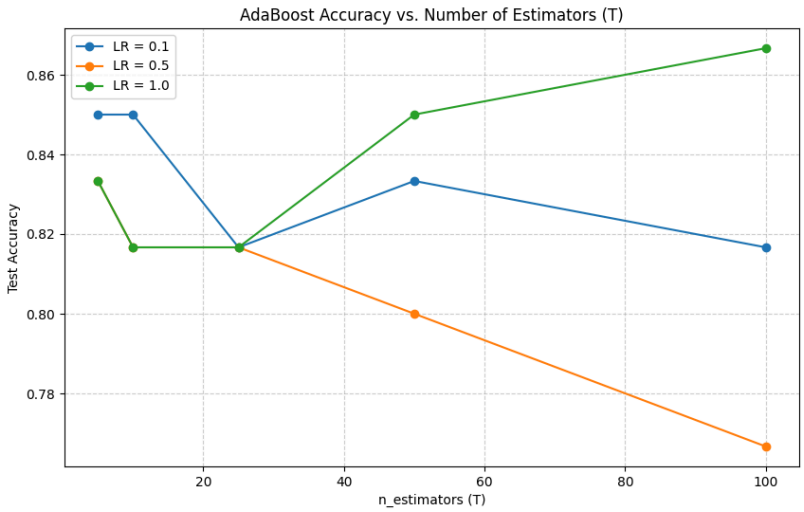
```
n_estimators_list = [5, 10, 25, 50, 100]
learning_rate_list = [0.1, 0.5, 1.0]

results = pd.DataFrame(columns=["n_estimators", "learning_rate", "test_accuracy"])
best_acc = 0
best_config = {}

plt.figure(figsize=(10, 6))

for lr in learning_rate_list:
    accuracy_scores = []

    for n_est in n_estimators_list:
        ada_clf = AdaBoostClassifier(
            estimator=DecisionTreeClassifier(max_depth=1, random_state=RANDOM_STATE),
            n_estimators=n_est,
            learning_rate=lr,
            random_state=RANDOM_STATE,
        )

        ada_clf.fit(X_train, y_train)
        test_acc = ada_clf.score(X_test, y_test)

        accuracy_scores.append(test_acc)

        results.loc[len(results)] = [n_est, lr, test_acc]

        if test_acc > best_acc:
            best_acc = test_acc
            best_config = {"n_estimators": n_est, "learning_rate": lr}

    plt.plot(n_estimators_list, accuracy_scores, marker="o", label=f"LR = {lr}")

plt.title("AdaBoost Accuracy vs. Number of Estimators (T)")
plt.xlabel("n_estimators (T)")
plt.ylabel("Test Accuracy")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
plt.savefig("adaboost_accuracy_vs_estimators.png")

print("\nHyperparameter Search Results (Accuracy):")
print(
    results.pivot(
        index="n_estimators", columns="learning_rate", values="test_accuracy"
    ).round(4)
)
print(
    f"\nIdentified Best Configuration: n_estimators={best_config['n_estimators']},
learning_rate={best_config['learning_rate']} (Accuracy: {best_acc:.4f})"
)
print("-" * 50)

best_model = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=RANDOM_STATE),
    n_estimators=best_config["n_estimators"],
    learning_rate=best_config["learning_rate"],
    random_state=RANDOM_STATE,
)
```

```
best_model.fit(X_train, y_train)

staged_train_errors = 1 - np.array(list(best_model.staged_score(X_train, y_train)))
staged_iterations = np.arange(1, len(staged_train_errors) + 1)

plt.figure(figsize=(8, 5))
plt.plot(staged_iterations, staged_train_errors, marker="o", color="purple")
plt.title("Weak Learner Ensemble Error (1 - Accuracy) vs. Iteration")
plt.xlabel("Boosting Iteration (t)")
plt.ylabel("Ensemble Training Error")
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
plt.savefig("adaboost_ensemble_error_vs_iteration.png")

final_weights = best_model.estimator_weights_

estimator_weights = best_model.estimator_weights_

plt.figure(figsize=(8, 5))
plt.plot(estimator_weights, marker="o")
plt.title("Estimator Weights (alpha_t) Across Boosting Iterations")
plt.xlabel("Iteration")
plt.ylabel("Alpha (Estimator Weight)")
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
plt.savefig("adaboost_estimator_weights.png")

importance = best_model.feature_importances_

feature_names_list = list(preprocessor.get_feature_names_out())

feature_importance_df = pd.DataFrame({
    "feature": feature_names_list,
    "importance": importance,
})
feature_importance_df = feature_importance_df.sort_values(
    by="importance", ascending=False
)

plt.figure(figsize=(12, 6))
plt.bar(
    feature_importance_df["feature"][:10],
    feature_importance_df["importance"][:10],
    color="skyblue",
)
plt.xticks(rotation=45, ha="right")
plt.title("Top 10 Feature Importance from AdaBoost Model")
plt.ylabel("Relative Importance")
plt.tight_layout()
plt.show()
plt.savefig("adaboost_feature_importance.png")

top_5_features = feature_importance_df.head(5)
print("\nTop 5 Most Important Features:")
print(top_5_features)
```

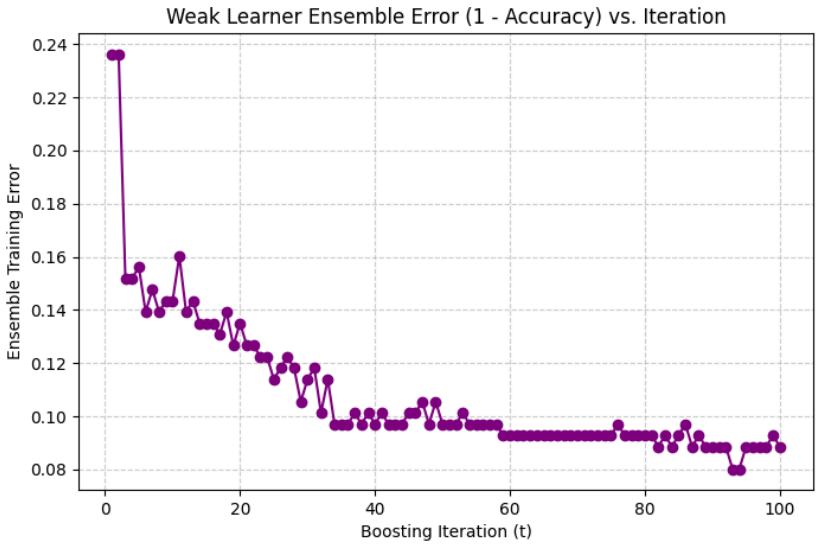
Output

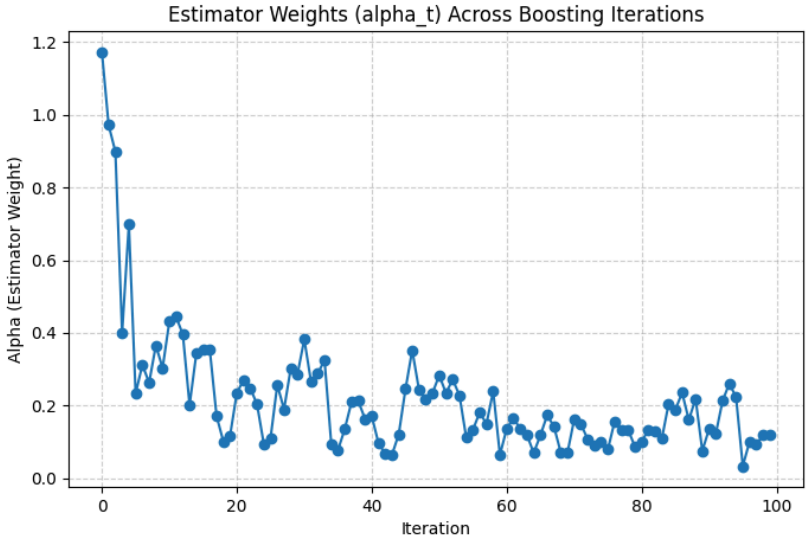
```
  Processed Data Shapes: Train (237, 28), Test (60, 28)
Training Accuracy: 0.7637
```

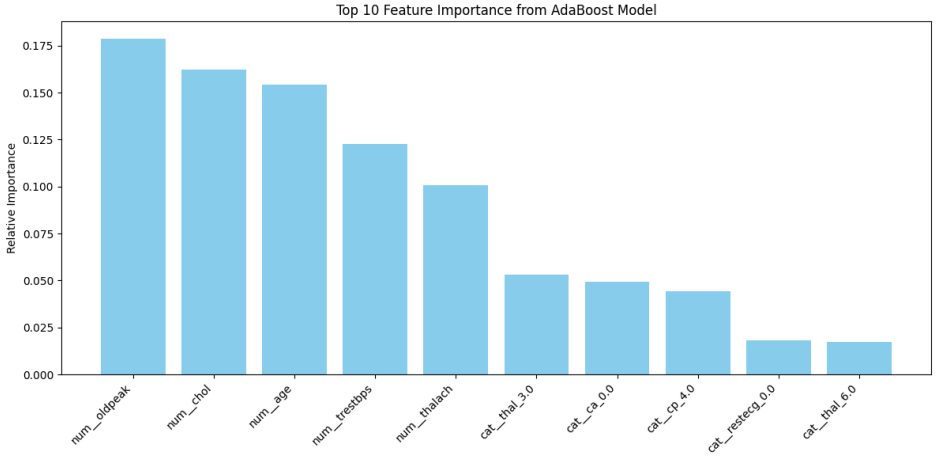
```
Test Accuracy:     0.7667

Confusion Matrix (Test):
 [[28  4]
 [10 18]]

Classification Report (Test):
              precision    recall  f1-score   support

           0       0.74      0.88      0.80        32
           1       0.82      0.64      0.72        28

    accuracy                           0.77        60
   macro avg       0.78      0.76      0.76        60
weighted avg       0.77      0.77      0.76        60
```




```
Hyperparameter Search Results (Accuracy):
learning_rate     0.1     0.5     1.0
n_estimators
5.0            0.8500  0.8333  0.8333
10.0           0.8500  0.8167  0.8167
25.0           0.8167  0.8167  0.8167
50.0           0.8333  0.8000  0.8500
100.0          0.8167  0.7667  0.8667

Identified Best Configuration: n_estimators=100, learning_rate=1.0 (Accuracy: 0.8667)
--------------------------------------------------
```







Top 5 Most Important Features:

	feature	importance
4	num__oldpeak	0.178848

```
2      num__chol    0.162175
0       num__age    0.154201
1  num__trestbps    0.122731
3   num__thalach    0.100626
```